# LINFO1341 : Projet SRTP

Mbengue Abdou

Yamdjieu Isaac

Bortoluzzi Cesar

# Architecture du client

- Envoie d’une requête HTTP simplifié au serveur avec un timestamp initial.
- Réception du packet et décodage avec protocole (si paquet corrompu, ignoré)
- Ecriture si paquet attendu, stockage dans buffer sinon
- ACK renvoyé si pas de perte, SACK sinon
- Si timeout : retransmet la requête GET si le transfert n’a pas commencé, envoie le dernier ack sinon

# Architecture du serveur

- Liste des Clients avec chacun leurs états
- Reçoit un paquet et vérifie si le client est présent dans la liste
- Si présent, traite ACK/SACK
- Sinon : vérifie la requête DATA, ouvre un fichier et ajoute le client à la liste
- Envoie des données tant que fenêtre pas pleine et fchier pas fini
- Si timeout : si l’envoie n’est pas fini, envoie les sauvegardes du packets; retransmet le paquet de fin si pas encore acquitté
- Transfert considéré terminé 3 secondes après l’envoie du paquet de fin
- A la fin du transfert, envoie un packet vide, attends le ACK et nettoie le client de la liste

# Timestamp

- Utilisé pour calculer le RTT (temps que met une requête pour être traité et que la réponse revienne) et définir un timeout.
- Initialisé comme le temps actuel multiplié par 1000 modulo 32 bits (2 exposant 32).
- Permet de toujours savoir le moment où la requête a été envoyé.

# Retransmission timeout

- Adaptatif, maximum entre 0.5 et 2.5 fois RTT estimé (0.875 * ancien RTT + 0.125 * mesure).
- Permet d’éviter les retransmissions inutiles tout en étant réactif aux variations réseaux.

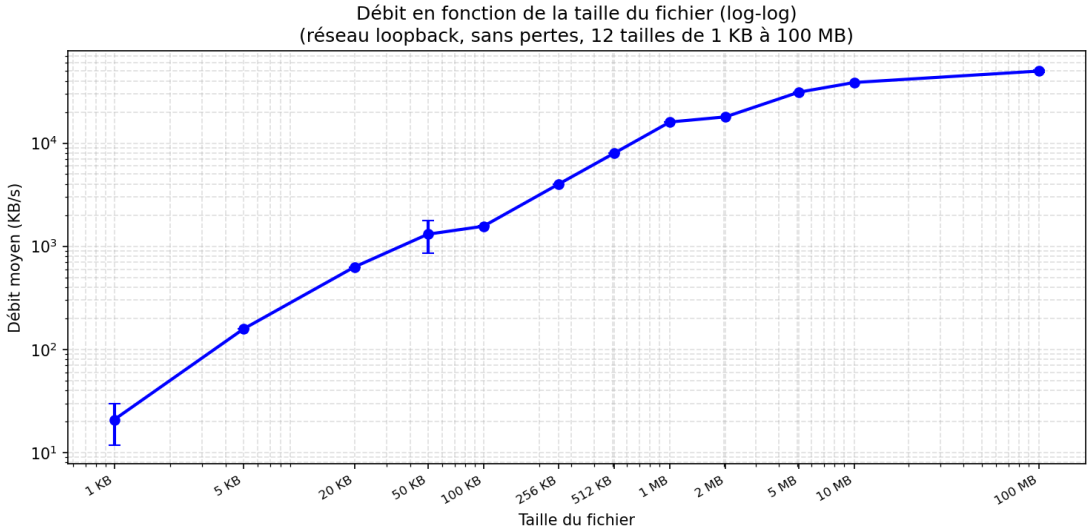
# Partie critique

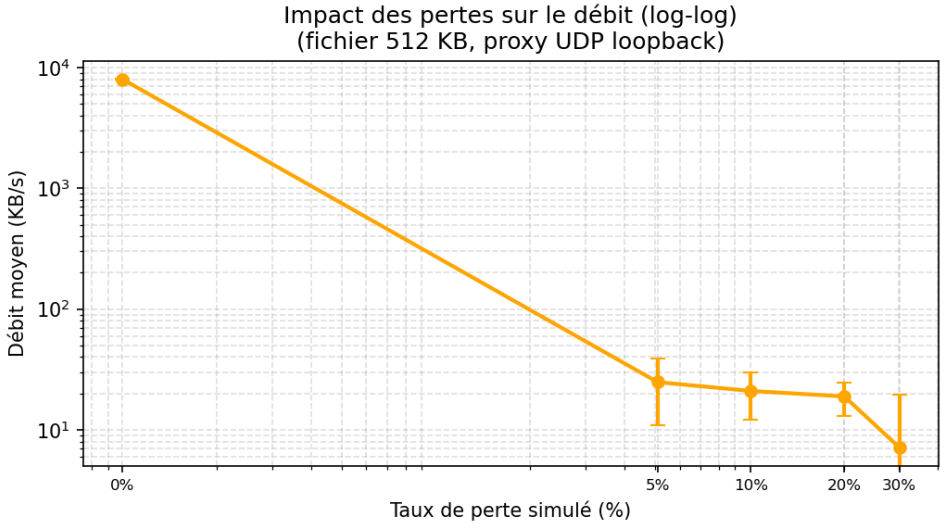
- La partie critique de notre implémentation affectant la vitesse de transfert est la gestion de la fenêtre et du buffer.
- Taille maximale de la fenêtre = 63.
- Retransmissions et traitement des ACK/SACK plus complexes
- Trop de pertes et de désordre dans les envois -> grand nombre de retransmissions

# Performances

- Mesures : débit (bytes/sec) vs temps de transfert
- Scénarios : réseau parfait, fichiers de tailles variés, pertes
- Méthodologie : Comparaison **MD5** (intégrité), Tests automatisés avec pytest, Proxy UDP pour interopérabilité, fichiers déterministes, environnement contrôlé (loopback)

# Graphiques de performances



# Tests

- **Tests unitaires :** empackage / depackage, CRC, SACK encode/decode
- **Tests d'intégration :** client ↔ serveur, fichiers de tailles variées
- **Tests d'interopérabilité :** avec implémentation de référence
- **Tests robustesse :**pertes, corruption, dedoublement, multi-clients

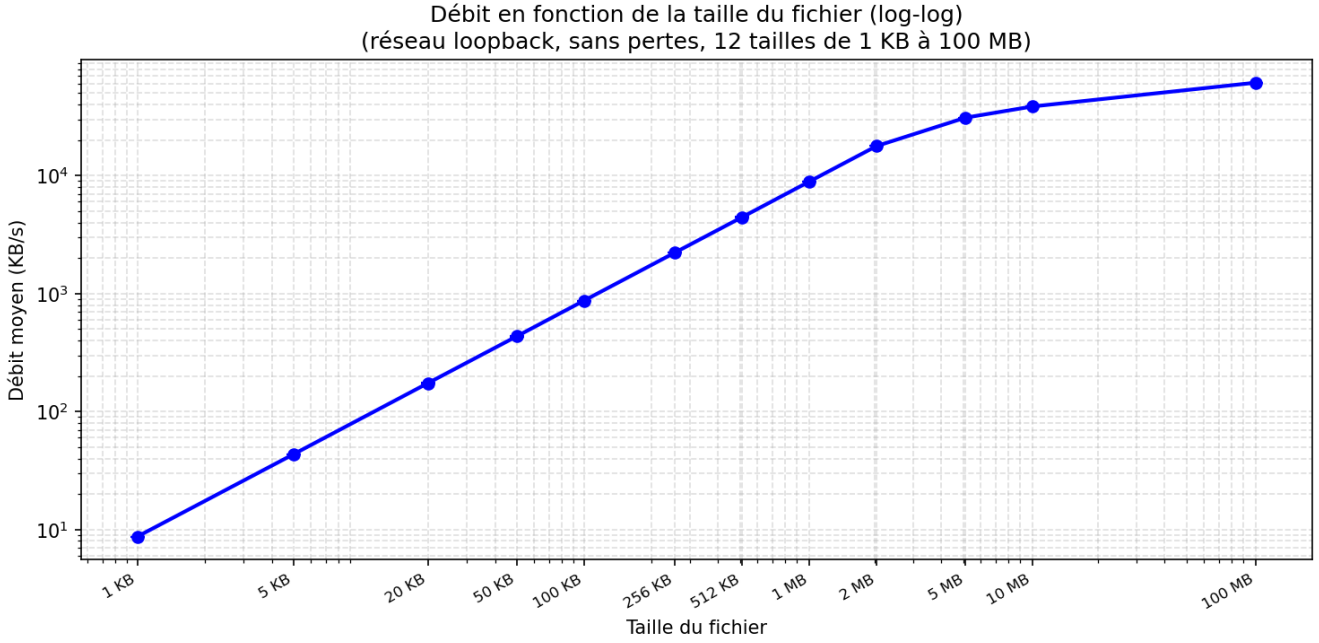
# Acquittements sélectifs

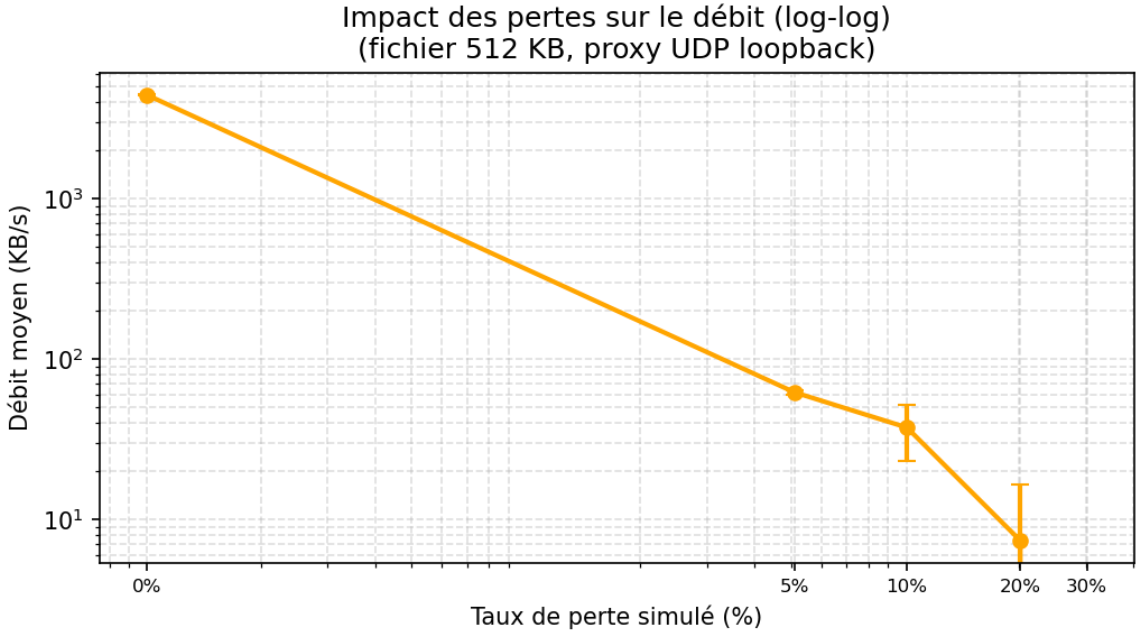
- Retransmissions ciblées : plus efficace en cas de perte, moins de trafic inutile
- Gains : amélioration du débit, réduction latence, plus efficace face aux pertes (moins de retransmissions)

# Interopérabilité

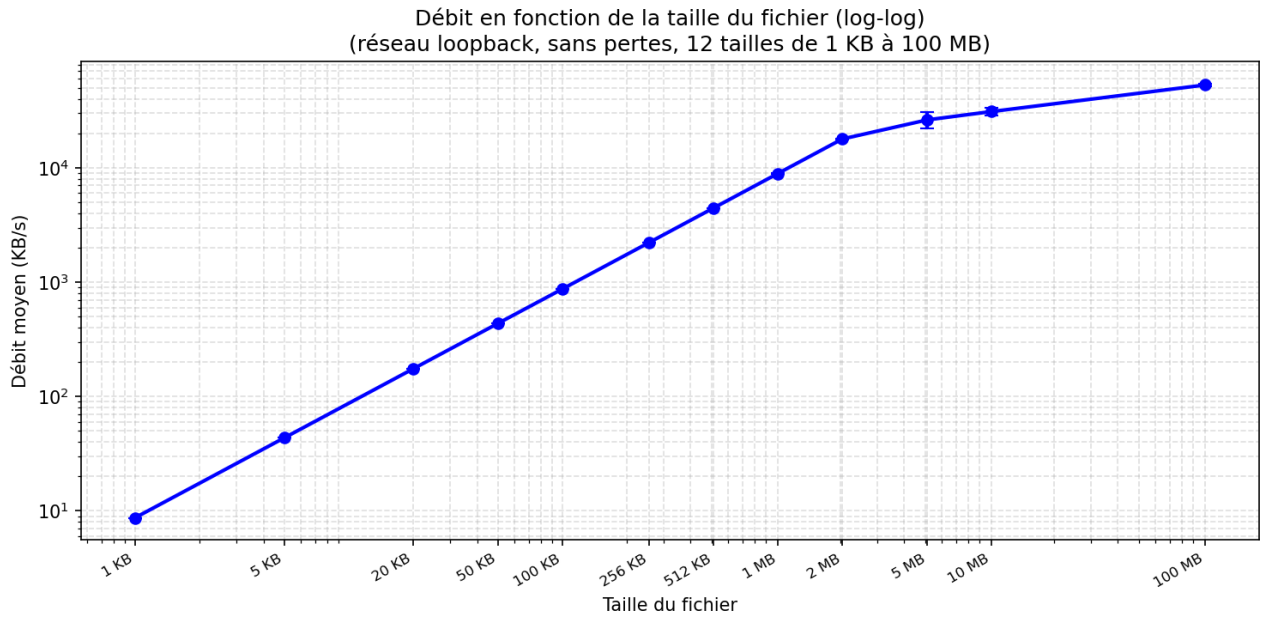
- Tests avec groupes 42 et 61
- Notre serveur/client fonctionnait avec ceux de ces groupes
- **Inconvénients**: problèmes dans certains cas de transfert : avec beaucoup perte, fichiers volumineux

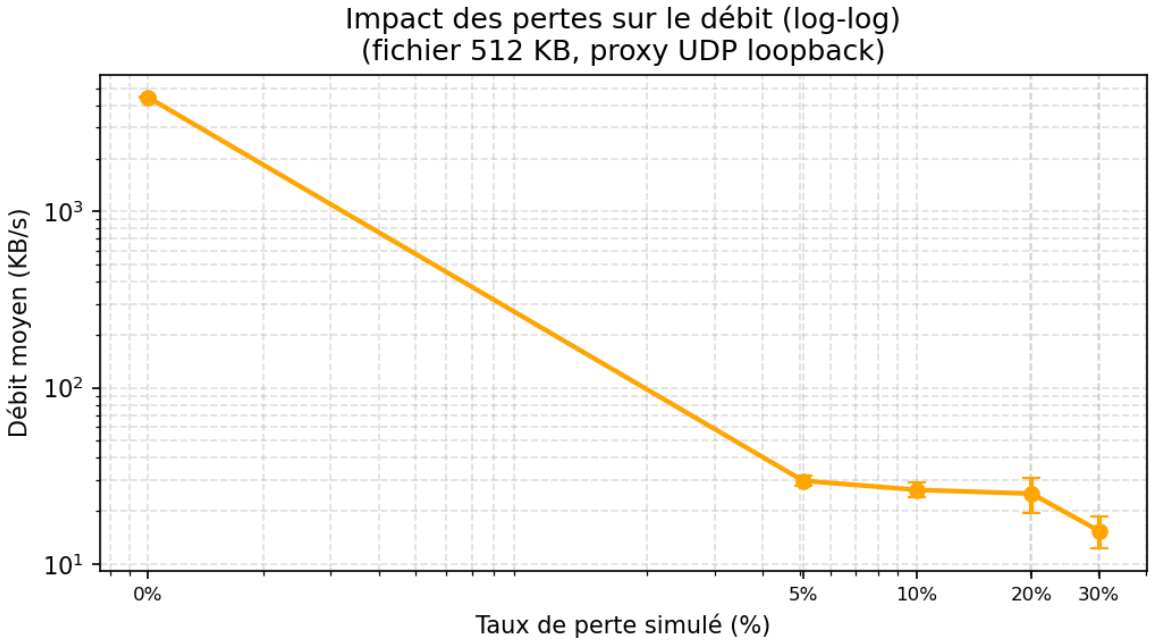
# Groupe 42 : client



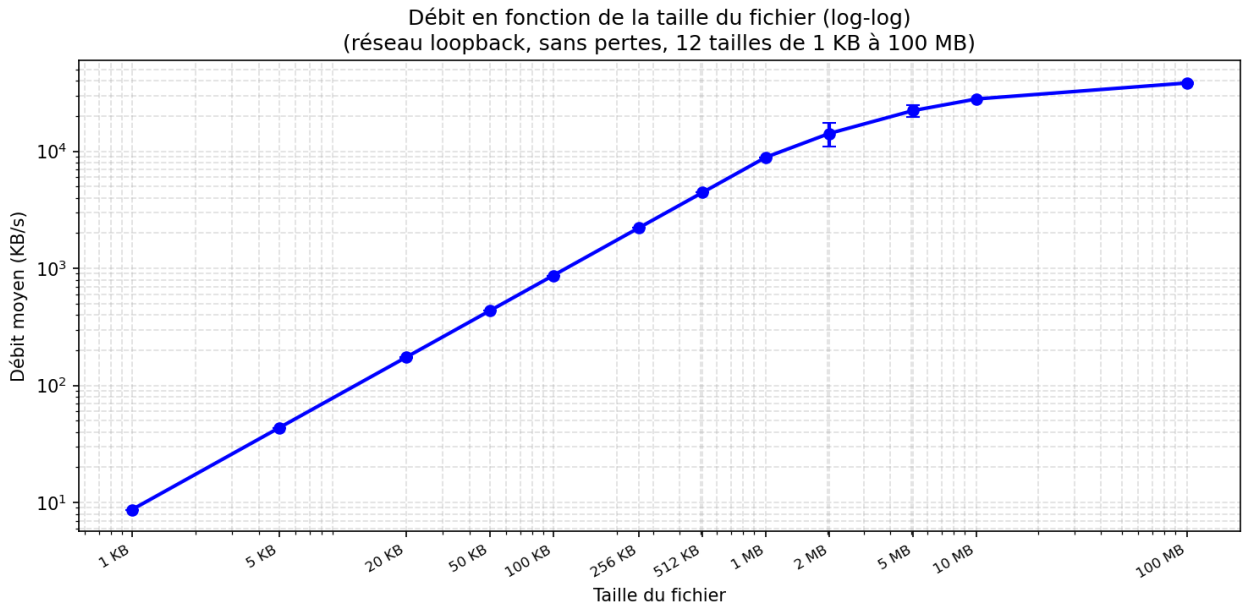



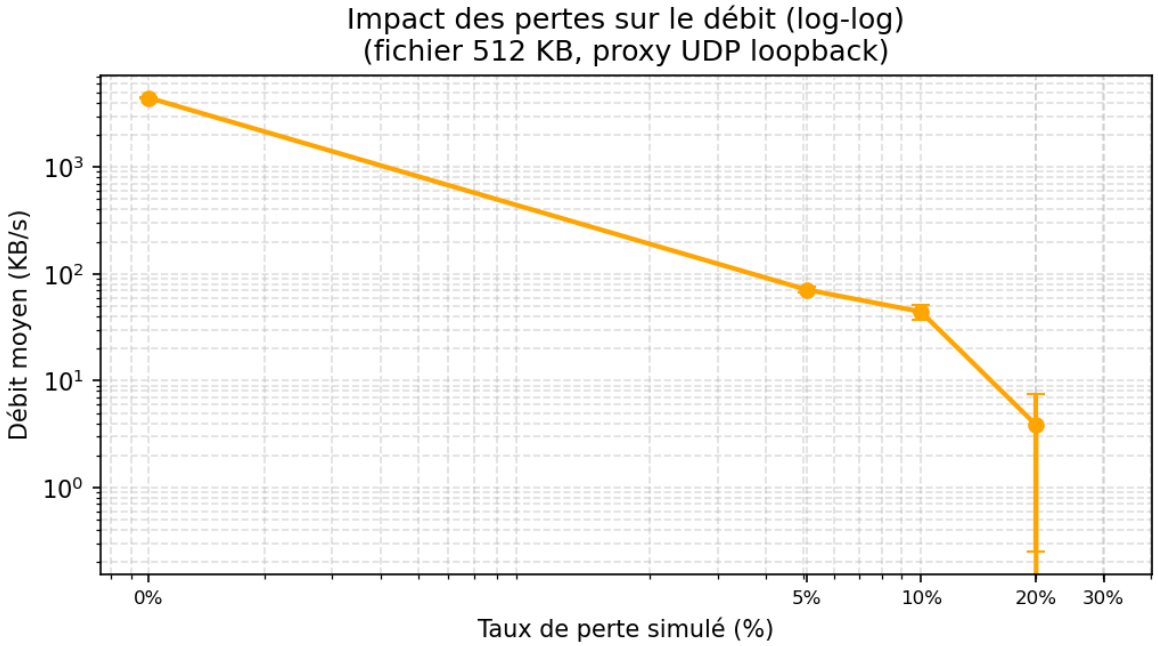
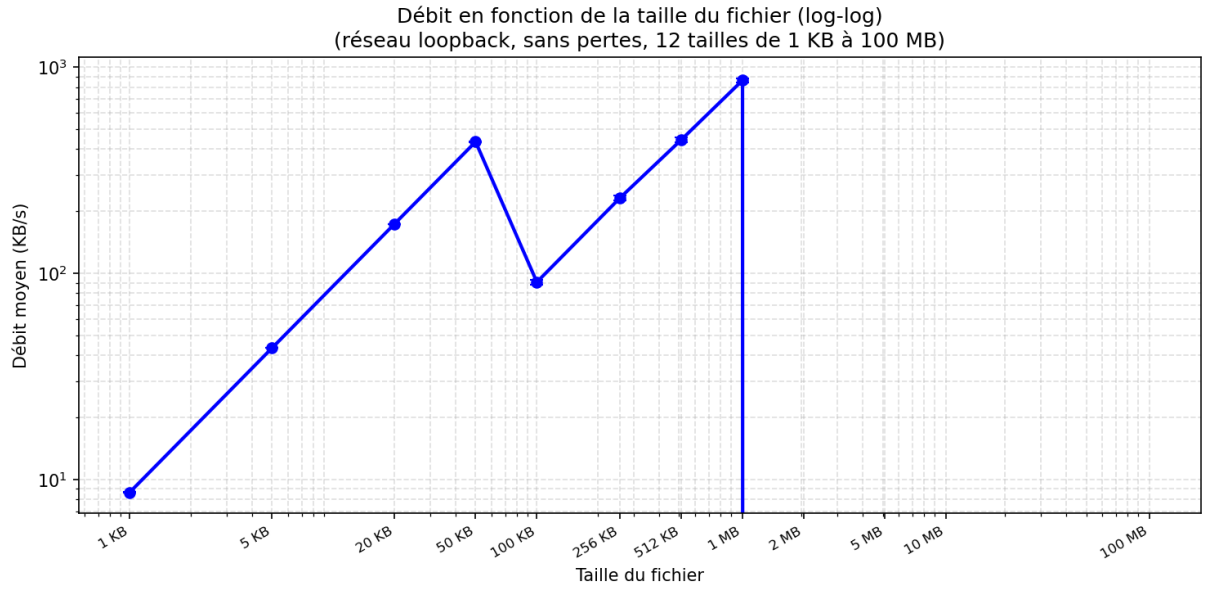
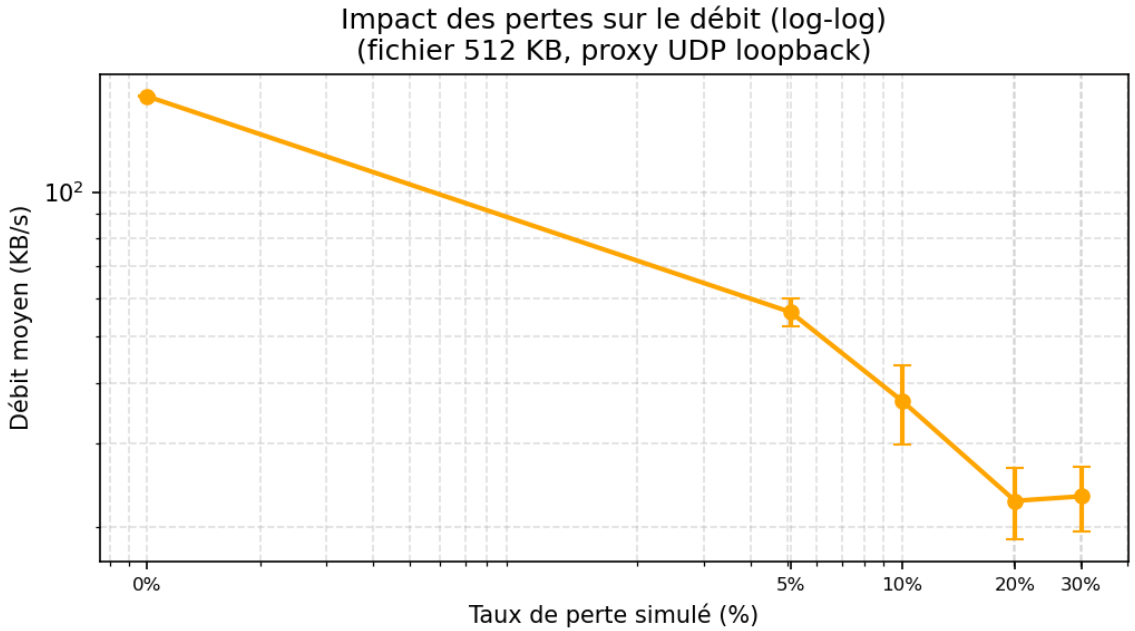
# Groupe 42 : serveur



# Groupe 61 : client






# Groupe 61 : serveur